



POO - Programação Orientada a objeto



COMPONENTE CURRICULAR:	PROGRAMAÇÃO ORIENTADA A OBJETOS I
CARGA HORÁRIA:	60H

OBJETOS DO CONHECIMENTO:

1. Paradigma Orientado a Objetos: Conceitos fundamentais: Classes (modelo/plantilla) e Objetos (instâncias). Atributos (estado) e Métodos (comportamento). Construtores e Destrutores. Encapsulamento: modificadores de acesso (public, private, protected), métodos getters e setters. Sobrecarga (Overloading) de construtores e métodos.
2. Relacionamentos e Herança: Relacionamentos entre classes: associação (unidirecional e bidirecional), agregação (todo-parte sem dependência existencial), composição (todo-parte com dependência existencial). Herança: subclasses e superclasses, reutilização de código, especialização e generalização. Polimorfismo: sobrescrita (Override) de métodos, classes e métodos abstratos. Classes Abstratas e Interfaces: conceitos, diferenças e aplicações.
3. Interfaces e Abstração: Definição e implementação de Interfaces (contrato de métodos). Uso de interfaces para definir comportamentos comuns entre classes não relacionadas. Programação para interfaces vs. programação para implementação.
4. Fundamentos de Coleções: Introdução às coleções (Listas, Conjuntos, Mapas). Uso de Arrays de Objetos. Iteração sobre coleções: loops for-each, iteradores.
5. Tecnologias e Ferramentas: Linguagem: Java (principal) ou C#. IDEs: IntelliJ IDEA, Eclipse, NetBeans, VS Code. Controle de versão: Git para versionamento de código.

COMPETÊNCIAS:

- Compreender e aplicar os pilares da Programação Orientada a Objetos (POO): encapsulamento, herança, polimorfismo e abstração.
- Projetar e implementar sistemas utilizando classes, objetos e relacionamentos.
- Utilizar herança e polimorfismo para promover a reutilização e a extensibilidade do código.
- Implementar abstrações através de classes abstratas e interfaces.
- Modelar e implementar relacionamentos de associação, agregação e composição entre classes.
- Aplicar coleções para armazenar e manipular grupos de objetos de forma eficiente.

HABILIDADES:

- Definir classes que representem entidades do mundo real com seus atributos e comportamentos.
- Criar objetos e manipular seus estados por meio de métodos (construtores, getters, setters).
- Aplicar o encapsulamento para proteger os dados internos de uma classe.
- Implementar estruturas de herança para criar hierarquias de classes (subclasses e superclasses).
- Utilizar o polimorfismo para tratar objetos de diferentes classes de forma uniforme.
- Modelar e implementar relacionamentos de associação e composição entre classes.

REFERÊNCIAS BIBLIOGRÁFICAS BÁSICA:

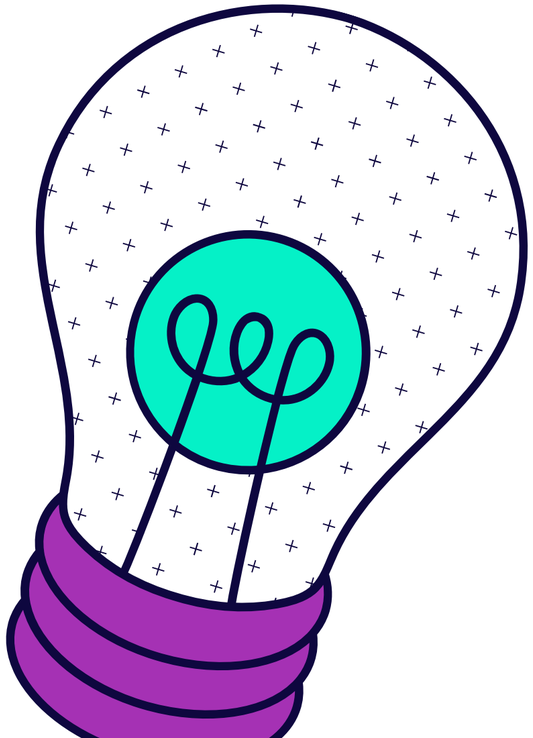
DEITEL, Paul; DEITEL, Harvey. Java: como programar. 10. ed. São Paulo: Pearson, 2017.
LARMAN, Craig. Utilizando UML e Padrões. 3. ed. Porto Alegre: Bookman, 2007.
SIERRA, Kathy; BATES, Bert. Use a cabeça! Java. 2. ed. Rio de Janeiro: Alta Books, 2007.
BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java. 4. ed. São Paulo: Pearson, 2009.
MARTIN, Robert C. Código limpo: habilidades práticas do Agile software. Rio de Janeiro: Alta Books, 2009.

REFERÊNCIAS BIBLIOGRÁFICAS COMPLEMENTAR:

HORSTMANN, Cay S.; CORNELL, Gary. Core Java, volume I: fundamentos. 10. ed. São Paulo: Pearson, 2017.
SCHILDT, Herbert. Java para iniciantes. 6. ed. Porto Alegre: Bookman, 2015.
BLOCH, Joshua. Effective Java. 3. ed. São Paulo: Addison-Wesley, 2018.
FREEMAN, Eric; FREEMAN, Elisabeth; ROBSON, Elisabeth; SIERRA, Kathy; BATES, Bert. Use a cabeça! Padrões de projeto. 2. ed. Rio de Janeiro: Alta Books, 2014.
GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.



Você sabe o que é Programação?

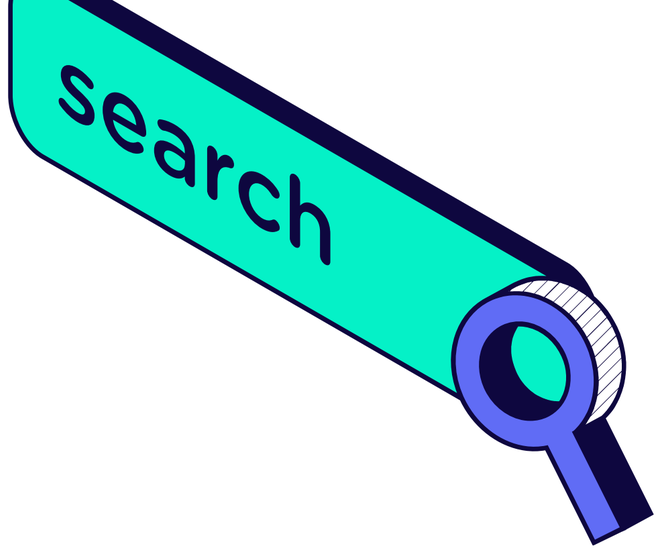


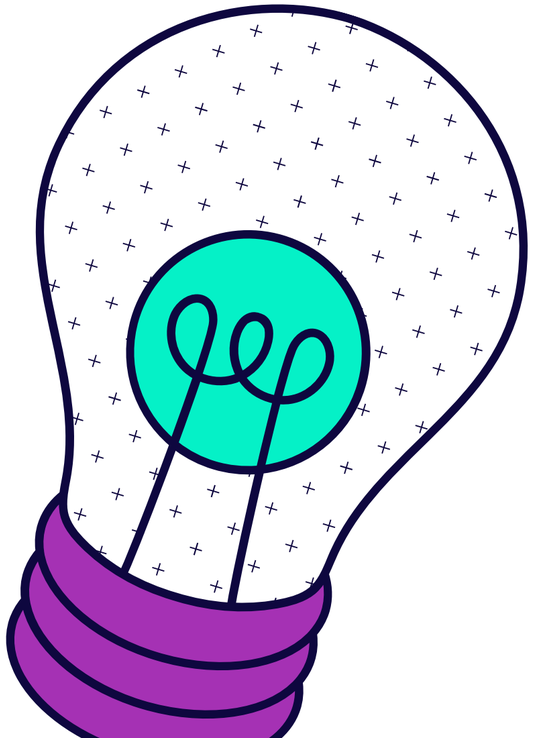


O que é programação ?

- De forma bem simples:
  - **Programação é dar instruções para o computador realizar uma tarefa.**
- O computador não “entende” o que queremos fazer sozinho. Nós precisamos escrever comandos em uma linguagem de programação para dizer a ele o que fazer.
- Programação **é o processo de criar instruções para que o computador realize tarefas e resolva problemas.** Para isso, utilizamos linguagens de programação, como Dart, Java, Python e JavaScript, que permitem transformar nossas ideias em código. Em resumo, programar é ensinar o computador o que fazer, como fazer e em que ordem executar cada ação.

Você sabe o que é POO?







O que é programação orientada a objetos?

- Programação orientada a objetos (POO) é uma forma de estruturar o código que simula o mundo real por meio de “**objetos**”. Assim, cada objeto representa uma entidade com características (atributos) e comportamentos (métodos). Essa forma de pensar facilita a construção de sistemas mais organizados, modulares e reutilizáveis.

- Quando alguém pergunta o que é programação orientada a objetos, a resposta envolve mais do que uma técnica de escrita. Trata-se de uma lógica que ajuda o desenvolvedor a lidar com a complexidade dos sistemas de forma mais clara e estruturada.



O que é programação orientada a objetos?

- Ela permite agrupar dados e comportamentos em estruturas únicas (as classes), que funcionam como moldes para criar os objetos que interagem entre si.

Essa abordagem estimula boas práticas de desenvolvimento, como a separação de responsabilidades e a reutilização de código.

- Além disso, prepara o programador para atuar em ambientes colaborativos e projetos de larga escala — onde organização e clareza fazem toda a diferença.
- De maneira mais simples: POO (Programação Orientada a Objetos) é uma forma de programar que **organiza** o código em **objetos**, representando coisas do mundo real. **POO é uma maneira de organizar o código usando objetos que possuem características e ações.**

Para que serve a programação orientada a objetos?



- A Programação Orientada a Objetos (POO) serve para organizar o código de forma mais estruturada, reutilizável e fácil de manter, principalmente quando o sistema começa a ficar grande.
    - **“POO serve para transformar um problema do mundo real em partes organizadas dentro do código.”**
- **A POO serve para criar sistemas mais organizados, reutilizáveis, seguros e fáceis de manter, dividindo o programa em objetos que possuem dados e comportamentos.**
- **Pergunta:** “Se eu fosse criar um aplicativo para uma escola, quais objetos eu precisaria representar no código?”

Para que serve a programação orientada a objetos?



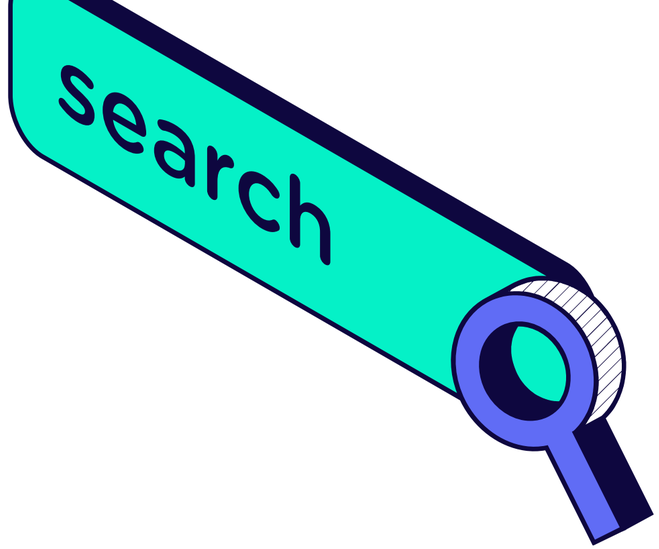
Paticamente em todo lugar onde há software.

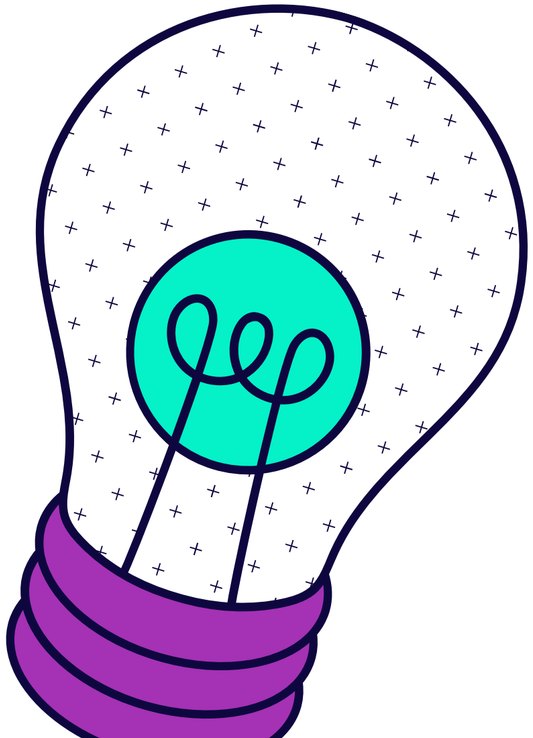
- A POO é aplicada em sistemas bancários, aplicativos de celular, plataformas de e-commerce, jogos, sistemas embarcados e muito mais.
- Essa estrutura orientada a objetos serve para organizar o código de forma lógica e eficiente, facilitando o desenvolvimento, a manutenção e a expansão dos sistemas ao longo do tempo.
- Com ela, é possível dividir um projeto em componentes independentes, o que reduz erros e aumenta a produtividade da equipe de desenvolvimento.

Principais Vantagens da POO:

- **Organização:** Separa o programa em partes fáceis de entender.
- **Reuso:** Evita reescrever o mesmo código várias vezes.
- **Manutenção:** Facilita encontrar e corrigir erros.
- **Trabalho em grupo:** Permite que várias pessoas criem partes diferentes ao mesmo tempo.

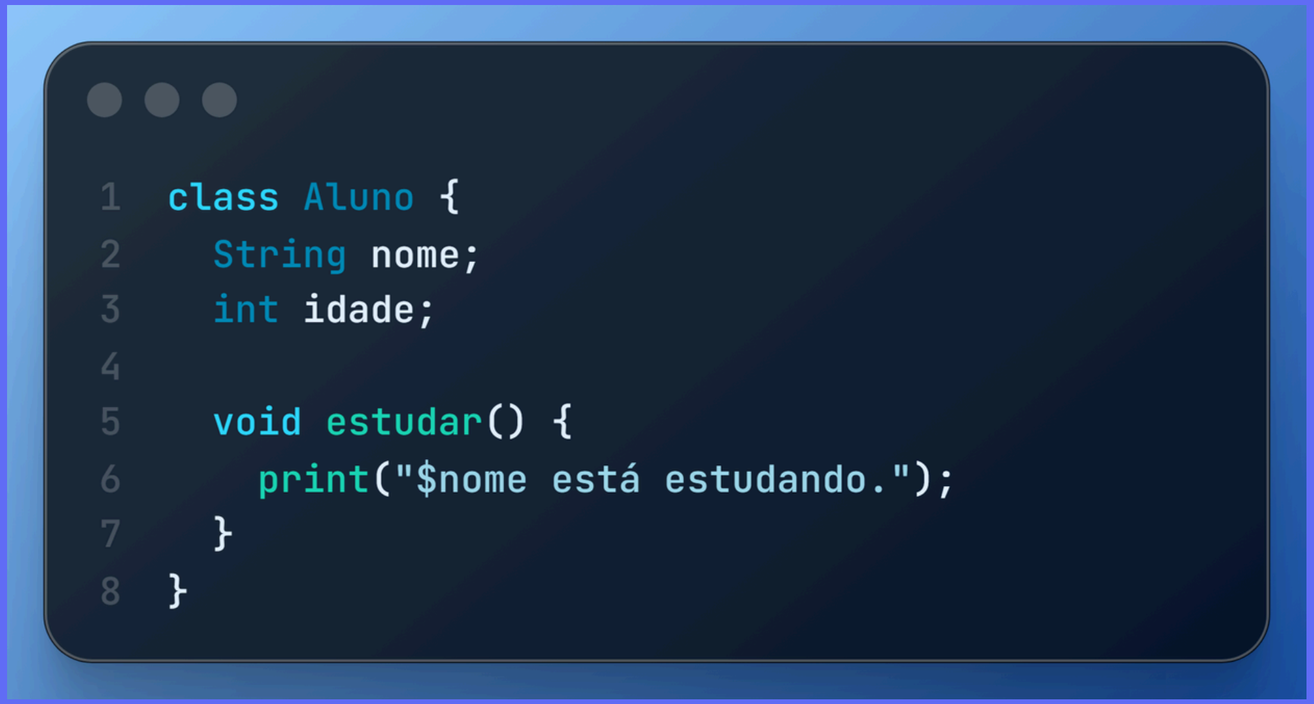
CLASSE
X
OBJETO





1. Classes

- Uma classe é uma estrutura utilizada para definir como determinados objetos serão criados e quais características e comportamentos eles terão.
- Podemos imaginar uma classe como uma planta de uma casa. A planta não é a casa em si; ela serve como um modelo que informa como a casa será construída.
- Na programação, a classe funciona de maneira semelhante: ela define quais dados e quais ações os objetos daquele tipo poderão possuir.



```
class Aluno {
  String nome;
  int idade;

  void estudar() {
    print("$nome está estudando.");
  }
}
```

Nesse exemplo, **Aluno** é uma **classe**.

Ela determina que um aluno terá:

- nome;
- idade;
- a ação estudar().

2. Objetos

- Um objeto é uma instância de uma classe, ou seja, algo que foi criado utilizando aquela classe como modelo.
- Se a classe é a planta de uma casa, os objetos são as casas construídas a partir daquela planta.
- Podemos criar vários objetos utilizando a mesma classe:

```
Aluno aluno1 = Aluno();
Aluno aluno2 = Aluno();
Aluno aluno3 = Aluno();
```

Agora temos três objetos diferentes da classe Aluno.

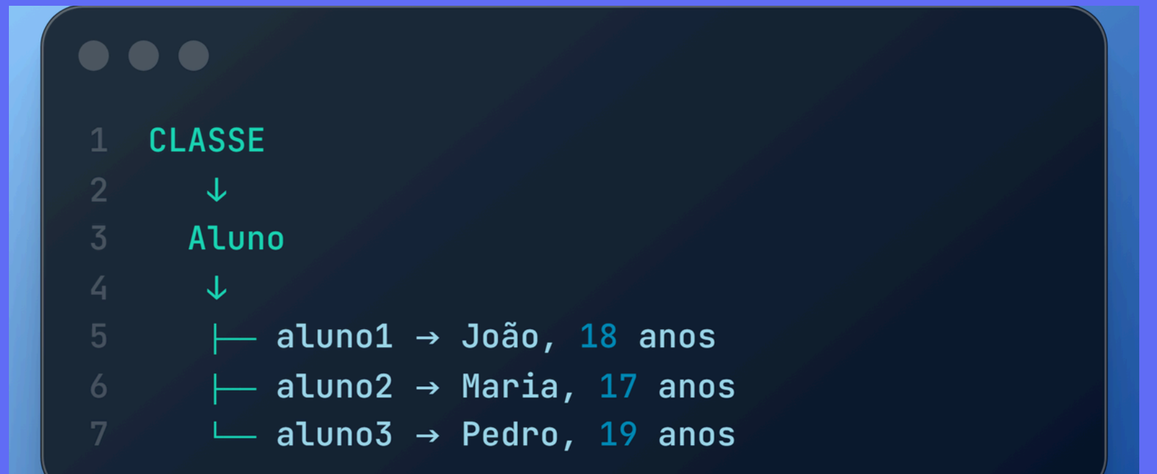
Podemos atribuir informações diferentes para cada um:

```
aluno1.nome = "João";
aluno1.idade = 18;

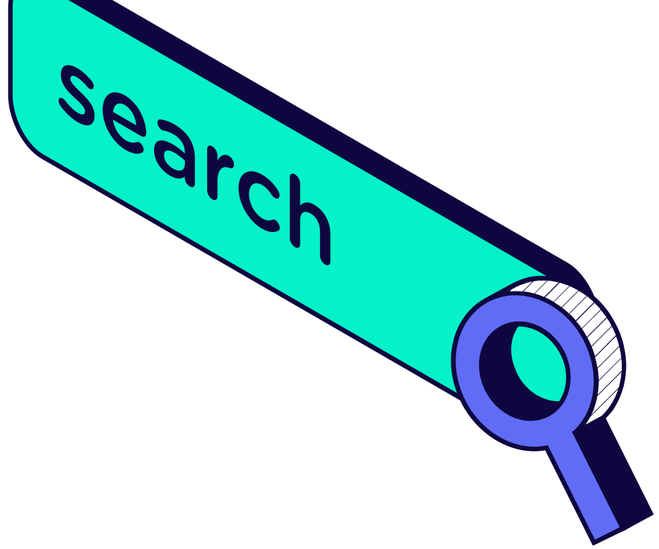
aluno2.nome = "Maria";
aluno2.idade = 17;

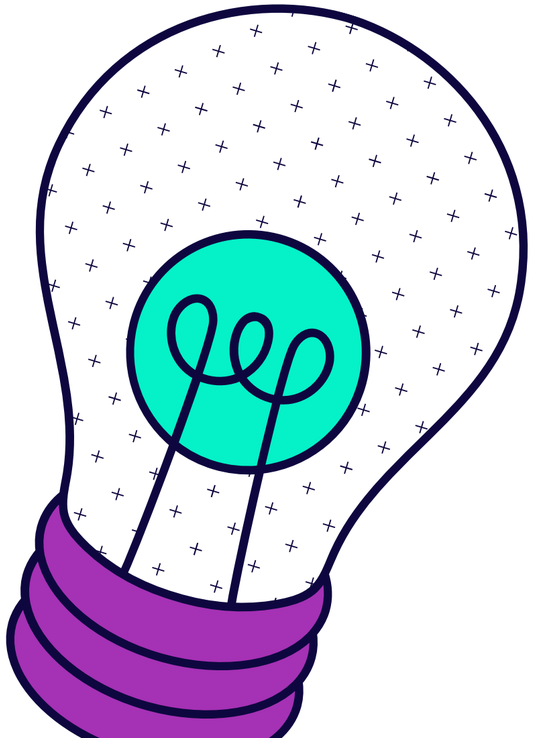
aluno3.nome = "Pedro";
aluno3.idade = 19;
```

Embora os três objetos pertençam à mesma classe, cada um pode possuir valores diferentes.



ATRIBUTOS X METODOS





3. Atributos

- Os atributos são as informações que descrevem um objeto.
- Eles representam as características ou o estado atual daquele objeto.
- No nosso exemplo:

```
String nome;
int idade;
```

são atributos da classe Aluno.

Um objeto pode possuir valores diferentes para esses atributos.

Por exemplo:

Os dois objetos possuem os mesmos atributos, mas seus valores são diferentes.

```
Aluno 1
Nome: João
Idade: 18

Aluno 2
Nome: Maria
Idade: 17
```

Outros exemplos de atributos

Em uma classe Carro:

```
class Carro {
  String marca;
  String modelo;
  String cor;
  int ano;
}
```

Os atributos seriam:

- marca, modelo, cor, ano

Eles representam as características do carro.

4. Métodos

Os métodos representam as ações ou comportamentos que um objeto pode executar.

Por exemplo, um carro pode:

- ligar;
- acelerar;
- frear;
- buzinar.

Podemos representar isso em Dart:

```dart
class Carro {
  String marca;
  String modelo;

  void ligar() {
    print("O carro foi ligado.");
  }

  void acelerar() {
    print("O carro está acelerando.");
  }

  void frear() {
    print("O carro está freando.");
  }
}
```

```dart
Nesse caso:

ligar()
acelerar()
frear()
```

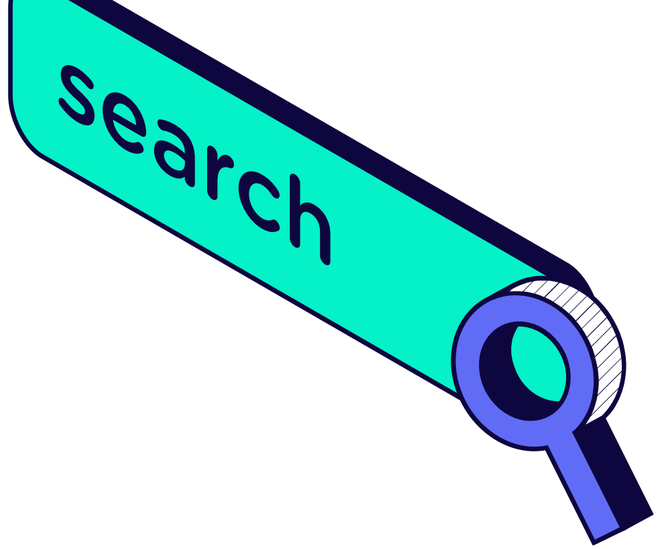
são métodos.

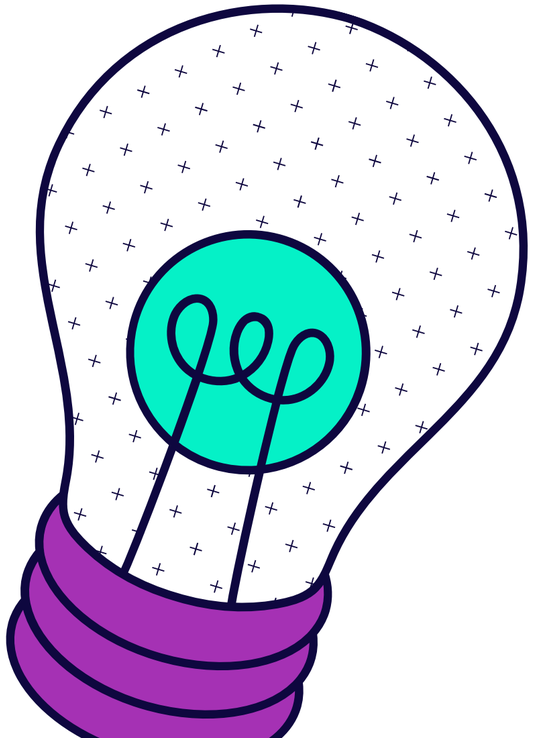
Uma forma simples de memorizar:

Atributos = aquilo que o objeto TEM.

Métodos = aquilo que o objeto FAZ.

Pilares da POO





Pilares da programação orientada a objetos

Compreender o que é programação orientada a objetos passa, inevitavelmente, por entender seus quatro pilares fundamentais. São eles que sustentam toda a lógica da POO e tornam o desenvolvimento mais modular, seguro e adaptável. São eles:

- Encapsulamento.
- Herança
- Polimorfismo
- Abstração



Pilares da programação orientada a objetos

Encapsulamento – Protegendo dados e controlando acessos

- Encapsular significa “**proteger**” os dados dentro de um objeto, evitando acessos diretos e não autorizados. Isso é feito controlando o que pode ou não ser visto ou alterado por outras partes do sistema.
- Com o encapsulamento, é possível evitar erros causados por manipulações externas indevidas e garantir maior segurança ao código.
- Encapsulamento = proteger os dados internos da classe e controlar como eles são acessados.



Pilares da programação orientada a objetos

Herança – Reutilizando estruturas e ampliando funcionalidades

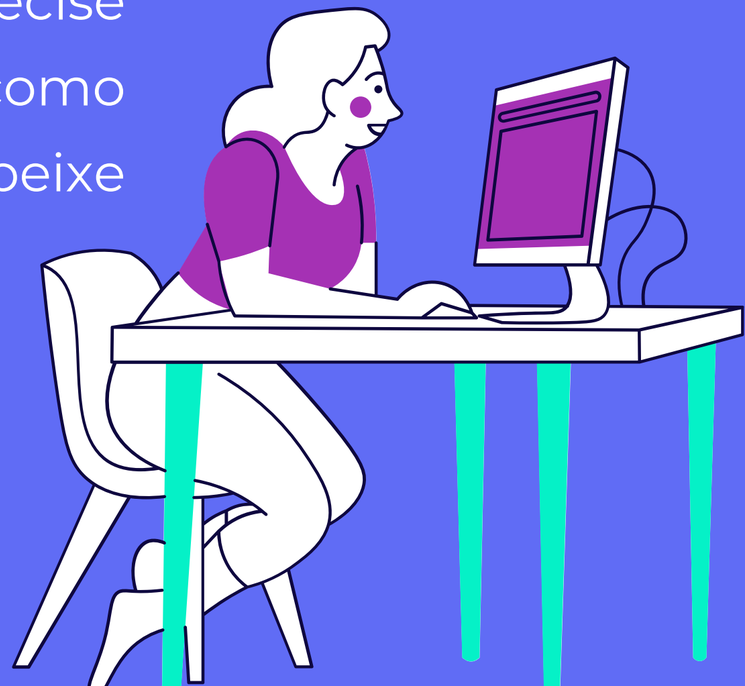
- A herança permite que uma classe “filha” aproveite atributos e comportamentos de uma classe “pai”. Isso evita repetição de código e torna o sistema mais coeso.
- Um exemplo simples: uma classe “Veículo” pode servir de base para as classes “Carro” e “Moto”, que herdariam seus comportamentos e adicionariam características específicas.



Pilares da programação orientada a objetos

Polimorfismo – Tornando o código mais flexível e adaptável

- Polimorfismo significa que objetos diferentes podem responder de formas distintas ao mesmo comando.
- Isso permite criar sistemas flexíveis, onde cada objeto sabe como se comportar sem que o programador precise escrever blocos de código específicos para cada um. É como dizer “mover” para um animal, e o cachorro correr, o peixe nadar e o pássaro voar.



Abstração – Focando no essencial e escondendo a complexidade

- A abstração permite representar apenas os aspectos mais relevantes de um objeto, escondendo detalhes desnecessários. Isso simplifica o código e facilita o entendimento do sistema.
- Pense em um botão de “comprar” em um site: o usuário vê uma função simples, mas por trás existem vários processos complexos que foram abstraídos.
- Dominar esses quatro pilares é o que transforma quem sabe o que é programação orientada a objetos em alguém que sabe aplicar o conceito na prática — com eficiência, clareza e visão de futuro.



🧠 **Onde está a abstração?**

O usuário precisa apenas fazer:

- maquina.prepararCafe();

Ele não precisa saber todos os processos necessários para preparar o café.

Por trás do método existem:

Aquecendo a água

↓

Passando pelo café

↓

Café pronto

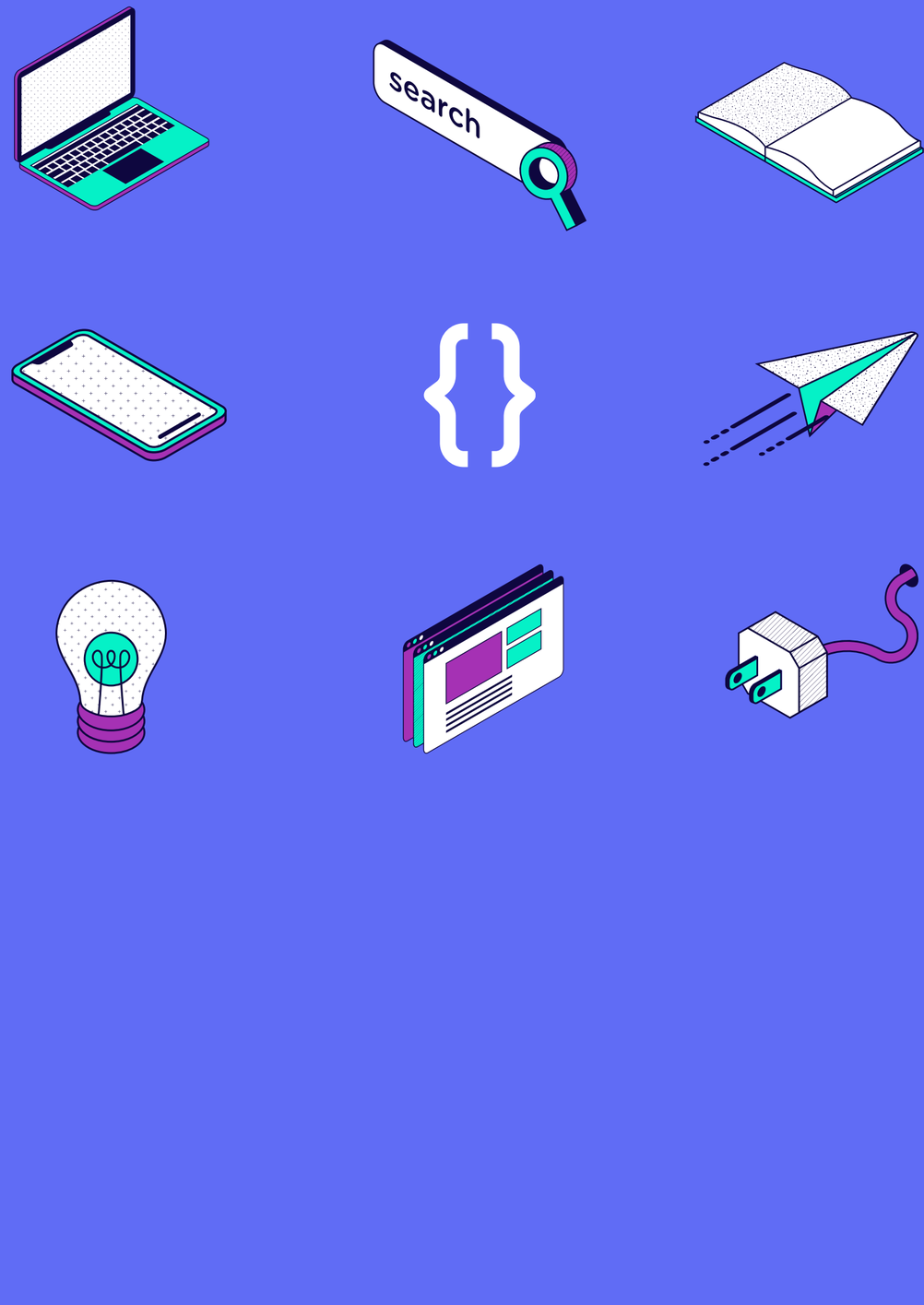
Pilares da programação orientada a objetos

Pilar	Ideia principal	Exemplo
Encapsulamento	Protege os dados	Conta bancária
Herança	Reutiliza características	Carro → Veículo
Polimorfismo	Mesmo comando, comportamentos diferentes	pagar() → Pix/Cartão
Abstração	Esconde a complexidade	prepararCafe()

Página de recursos

https://www.fiap.com.br/radar-tech/post/programacao-orientada-a-objetos-poo-um-guia-completo-sobre-seus-4-pilares-fundamentais-encapsulamento-heranca-polimorfismo-e-abstracao/

Remove-Item C:\ProgramData\chocolatey -Recurse -Force



Aprendendo na prática



Instale o SDK do Dart.

- **Para instalar o SDK do Dart, utilize o gerenciador de pacotes apropriado para sua plataforma de desenvolvimento.**
- **Para atualizar o SDK do Dart, execute o mesmo comando usado para instalar o SDK do Dart a partir do seu ade pacotes.**

Instale usando o Chocolatey

- **Para instalar o SDK do Dart, use o Chocolatey . O Chocolatey requer permissões elevadas .**

1. **Instale o Chocolatey.**
2. **Inicie o PowerShell com permissões elevadas.**
   a. **Set-ExecutionPolicy Bypass -Scope Process -Force; [System.Net.ServicePointManager]::SecurityProtocol = [System.Net.ServicePointManager]::SecurityProtocol -bor 3072; iex ((New-Object System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))**
   b. **PS C:\> choco install dart-sdk**
   c. **Alterar o caminho de instalação padrão. Por padrão, o Chocolatey instala o SDK em C:\tools\dart-sdk. Para alterar esse local, defina a ChocolateyToolsLocation variável de ambiente para o diretório de instalação desejado.**



